

Explicación de Proyecto: FrameffX

Nicolás Antón García

1. Introducción y Enfoque General del Proyecto

La edición de vídeo se ha convertido en una disciplina masiva. El boom de los formatos verticales cortos y los contenidos rápidos que inundan redes como TikTok o Instagram han despertado las ganas de editar en miles de personas. Sin embargo, los principiantes suelen encontrarse con un muro: tutoriales complejos y desordenados en internet y dificultades para conseguir recursos adecuados como presets o archivos de proyecto.

Por otro lado, los creadores de contenido que quieren enseñar se enfrentan a la fragmentación de su negocio. Suelen depender de una plataforma para vender archivos, otra para organizar sus clases en directo y una web propia como portfolio. Esto marea al cliente y devora los ingresos del creador en comisiones.

FrameffX nace para unificar ambos mundos en un único ecosistema digital, actuando como plataforma de aprendizaje en línea y mercado de archivos digitales (marketplace). La web ofrece un catálogo dinámico donde los mentores ofertan clases virtuales y mentorías en directo, mientras venden recursos descargables como preajustes de color, complementos de software (plugins) y guías de aprendizaje.

Para el alumno, la experiencia es realmente conveniente: puede asistir a una clase online y, desde el mismo perfil, adquirir los recursos exactos que el profesor usa en pantalla. Para el creador, la plataforma centraliza todo en un panel de control privado para supervisar ingresos, tutorías y ventas sin depender de informes externos.

El proyecto está dividido en aplicaciones internas independientes de Django (users, teachings, products, bookings y common). Esta separación modular garantiza que si surge un problema en el sistema de descargas del marketplace, el catálogo de formación y el acceso a las clases en directo sigan funcionando sin inmutarse, facilitando el mantenimiento técnico.

2. Arquitectura del Sistema y Stack Tecnológico

Un único servidor central procesa la lógica de negocio, consulta la base de datos y genera las páginas HTML que el usuario ve en pantalla. Incluye además una API REST paralela construida con Django Rest Framework, lista para que, si el negocio crece, se pueda lanzar una aplicación móvil nativa para iOS o Android que consuma los mismos datos sin tener que reescribir el backend.

Esta arquitectura unificada permite centralizar la seguridad y mantener un buen rendimiento. El viaje de una petición es sencillo:

- El usuario interactúa con la web y la solicitud llega al servidor web frontal Nginx.
- Si es un recurso estático (como un archivo CSS o una imagen), Nginx se lo entrega directamente al usuario para ahorrar recursos.
- Si la petición es dinámica (un inicio de sesión o una compra), Nginx la pasa a través de Gunicorn, que actúa como puente entre Nginx y Django.
- Gunicorn ejecuta el código de Django, que procesa las reglas de negocio, consulta la base de datos y devuelve la página ya construida.

En cuanto al diseño visual, el frontend está construido sobre un sistema de estilos propio, desarrollado completamente a medida con CSS puro y la tipografía Inter de Google Fonts. Para el renderizado automático de los formularios se emplea Django Crispy Forms, que genera la estructura HTML de cada campo desde el propio servidor. En el almacenamiento, el sistema usa SQLite en el ordenador de desarrollo por su ligereza, pero conmuta automáticamente hacia PostgreSQL 16 al subir al servidor real en producción, un motor preparado para soportar transacciones masivas y accesos concurrentes de muchos usuarios a la vez.

3. El Sistema Central de Configuración y Enrutamiento

Toda la inteligencia organizativa del proyecto se concentra en la carpeta raíz `FrameffX/`, gestionada por `settings.py` (reglas globales) y `urls.py` (el mapa de carreteras que dirige el tráfico web).

Para evitar el error común de dejar contraseñas escritas en el código o modificarlas a mano al cambiar de servidor, el sistema es sensible al entorno (environment-aware). El código fuente es idéntico en cualquier máquina; su comportamiento cambia dinámicamente leyendo un archivo oculto .env mediante la librería python-dotenv.

Cuando el modo de desarrollo se desactiva (DEBUG=False), Django activa de forma autónoma las defensas de producción: fuerza el redireccionamiento a HTTPS, bloquea cookies en canales inseguros y activa restricciones de enmarcación (X-Frame-Options) para evitar que incrusten nuestra web en sitios maliciosos.

Por su parte, urls.py organiza el tráfico de forma jerárquica. Si la dirección coincide con el frontend clásico, deriva al usuario hacia las plantillas HTML. Si empieza por /api/, traslada la solicitud al enrutador automático DefaultRouter de la API, que genera de forma autónoma los caminos para crear, editar o borrar elementos, reduciendo el código escrito.

4. Gestión de Usuarios y Autenticación Avanzada

Para mejorar la experiencia de usuario y eliminar barreras de entrada, el sistema base de Django se reescribió para eliminar el campo tradicional de username (que los usuarios suelen olvidar) y sustituirlo por el email como identificador único.

Además, se integró el inicio de sesión social con Google (OAuth 2.0) a través de Django-Allauth. Con un solo clic, el usuario está dentro. El sistema incluye una lógica de autoconexión: si te registraste a mano el mes pasado y hoy decides entrar con Google, la plataforma reconoce tu correo, fusiona ambas vías de entrada y te mantiene en tu cuenta de siempre, evitando perfiles duplicados.

A nivel de seguridad, las contraseñas jamás se guardan en texto plano. Se procesan mediante un algoritmo de encriptación unidireccional e irreversible, transformándolas en una cadena de caracteres indescifrable. El acceso a los datos está blindado mediante un modelo de usuario personalizado que hereda de AbstractBaseUser, asegurando que un alumno solo pueda ver su propio perfil e historial de compras.

5. El Catálogo de Clases Virtuales y Mentorías

El módulo de aprendizaje (teachings) se adapta a quien está mirando la pantalla. Si entra el administrador (is_staff), la base de datos le entrega el listado completo con botones rápidos para gestionarlo todo. Si entra un alumno, el sistema aplica un filtro temporal estricto: consulta la hora actual del servidor y descarta cualquier clase cancelada o que ya haya quedado atrás en el tiempo.

Para evitar errores humanos, la base de datos cuenta con una restricción que impide publicar dos clases con el mismo título exactamente a la misma hora. Además, detalles como la duración de la sesión se precálculan en el momento de su creación, evitando que el servidor tenga que recalcularlos en tiempo real cada vez que los usuarios entran al catálogo.

Si el alumno tiene la sesión iniciada, el backend extrae rápidamente sus reservas activas y transforma el botón de compra en un aviso de "Ya inscrito" para evitar duplicidades.

6. El Marketplace Digital y el Blindaje de Descargas

La tienda de recursos digitales gestiona la venta de archivos protegiendo la contabilidad y evitando la piratería masiva mediante dos mecanismos técnicos:

Normalización Financiera Histórica: Cuando un usuario realiza una compra, la información se divide en Pedido (cabecera del recibo) y DetallePedido (línea de facturación). El sistema copia el coste actual del producto y lo congela en el momento de la transacción. Si el administrador sube el precio de un preset al día siguiente, los recibos antiguos no se alteran, asegurando que las auditorías contables cuadren siempre al céntimo.

Cortafuegos de Descarga Ciega: Los archivos se guardan en un directorio privado. Al pulsar "Descargar", una vista de control verifica que el pedido esté pagado y que el usuario no haya superado el límite permitido (por ejemplo, un tope de 3 intentos). Si todo está en regla, el sistema registra la IP del usuario y le transmite el archivo de forma segura a través de Django. Esto enmascara por completo la ruta del servidor, y se complementa con una regla de denegación absoluta en Nginx que bloquea cualquier intento de acceso mediante URL directa.

7. El Motor de Reservas y Gestión de Transacciones

Este módulo controla la inscripción de los alumnos en las clases virtuales y gestiona sus estados (pendiente_pago, pendiente, confirmada, cancelada, caducada). Para evitar condiciones de carrera —como cuando queda una sola plaza libre y dos usuarios pulsan "Reservar" en el mismo milisegundo—, el corazón de la base de datos está blindado.

La vista de creación acepta únicamente peticiones de tipo POST y ejecuta tres validaciones encadenadas en el servidor:

- Bloquea la operación si el usuario es administrador.
- Comprueba que la clase exista, esté activa y pertenezca al futuro.
- Escanea que el usuario no tenga ya una reserva activa para esa misma sesión.

Si las validaciones dan luz verde, el sistema intenta insertar la reserva. Gracias a una restricción estructural incrustada en la base de datos, si dos peticiones chocan en el mismo instante, el motor físico frena la segunda transacción. La web captura esta colisión de forma silenciosa y le devuelve al usuario un mensaje amigable indicando que la plaza ya no está disponible. Para las cancelaciones, el sistema optimiza la consulta SQL para modificar únicamente el campo de estado, reduciendo la carga sobre la base de datos.

8. El Sistema de Pasarela de Pagos y Verificación por Webhooks

Para cumplir con las normativas internacionales de seguridad y no almacenar números de tarjeta de crédito en nuestro servidor, toda la responsabilidad financiera se delega en la pasarela de pagos Stripe. El sistema funciona mediante una doble vía de verificación:

Cuando un usuario va a comprar, el backend genera una sesión segura (Stripe Checkout Session) e inyecta un identificador interno ligado al pedido. El usuario introduce su tarjeta en el entorno cifrado de Stripe y, si el proceso tiene éxito, se activan dos mecánicas:

Vía Frontend (Síncrona): Stripe devuelve al usuario a nuestra web pasando un identificador de sesión en la URL, lo que permite a Django consultar la API de Stripe y mostrar una confirmación visual inmediata.

Vía Backend (Asíncrona y Crítica): De forma independiente al navegador, los servidores de Stripe realizan una llamada directa a un endpoint privado de nuestra web denominado Webhook. Esta llamada incluye una firma criptográfica que el servidor verifica usando una clave secreta. Si la firma es auténtica, el sistema localiza el pedido y actualiza su estado en la base de datos de forma automática, asegurando el servicio aunque el usuario cierre la pestaña antes de volver a la web.

9. Contenedores, Servidores y Seguridad en Producción

Para garantizar que el proyecto funcione de manera idéntica independientemente del servidor donde se aloje, toda la infraestructura está empaquetada utilizando Docker y Docker Compose, dividiendo la plataforma en tres servicios aislados dentro de una red virtual privada y cerrada:

PostgreSQL: Aloja el motor de la base de datos de forma aislada. Cuenta con un monitoreo de salud (Healthcheck) y solo atiende peticiones internas de la red de Docker.

Django + Gunicorn: Contiene el código de la aplicación. Al arrancar, el script de entrada (entrypoint.sh) vigila el estado de la base de datos; en cuanto está saludable, ejecuta las migraciones de las tablas de forma ordenada —migrando primero la aplicación users por dependencias entre módulos—, recopila los archivos estáticos en un directorio compartido y levanta Gunicorn con 4 procesos de trabajo en paralelo. El script también verifica e instala la cuenta del superusuario inicial si la base de datos está vacía, leyendo las credenciales desde las variables de entorno de forma segura.

Nginx: Es el único servicio expuesto al exterior (puertos 80 y 443). Recibe las peticiones globales, bloquea ataques, sirve los archivos estáticos a máxima velocidad y deriva las conexiones dinámicas hacia Gunicorn.

Para optimizar la seguridad y el rendimiento, se implementó una construcción en etapas (Multi-stage Dockerfile). En la primera fase, se descargan las herramientas pesadas de Linux para compilar las dependencias de Python. En la segunda fase, se trasladan únicamente los paquetes ya compilados a una imagen final basada en una distribución de Linux extremadamente ligera. Esto reduce el peso del contenedor y elimina los compiladores del sistema, impidiendo que un atacante los use si lograra comprometer el entorno.

Por último, el sistema integra un contenedor temporal de Certbot que se comunica con la entidad certificadora Let's Encrypt. Este contenedor completa el protocolo de validación ACME Challenge, confirmando la propiedad del dominio web. Una vez superada la prueba, Let's Encrypt genera los certificados de seguridad e indica a Nginx que active el cifrado de extremo a extremo HTTPS, garantizando que las contraseñas y datos transaccionales de los usuarios viajen por internet de forma totalmente confidencial. Finalmente, la web habrá sido desplegada con éxito.